

Politechnika Białostocka
Wydział Informatyki

Finansista PB

System obsługi wniosków o finansowanie
działalności studenckiej i doktoranckiej

Dokumentacja projektowa

Autorzy:

Jakub Matusiewicz
Jakub Borkowski

Przedmioty: Programowanie aplikacji WWW w języku Java, Systemy Baz Danych

Białystok, 14 czerwca 2026

Spis treści

1	Wstęp	3
1.1	Cel projektu	3
1.2	Zakres	3
1.3	Zespół i podział pracy	3
2	Wymagania	3
2.1	Wymagania funkcjonalne	3
2.2	Wymagania niefunkcjonalne	4
3	Architektura systemu	4
3.1	Stos technologiczny	5
4	Model danych	6
4.1	Diagram encji	6
4.2	Główne tabele	6
4.3	Wartości osadzone (<i>JPA Embeddables</i>)	7
4.4	Klucze hybrydowe	7
4.5	Migracje Flyway	7
5	Logika w bazie danych	8
5.1	Maszyna stanów wniosku	8
5.2	Pakiet <code>finansista_pkg</code> (V5)	8
5.3	Triggery (V4)	8
5.4	Maszyna stanów (V7)	9
5.5	Widoki (V3)	9
5.6	Indeksy wydajnościowe (V6)	9
5.7	Wyniki testów wydajnościowych (1 k – 1 M)	10
6	REST API	11
6.1	Najważniejsze endpointy	12
6.2	Specyfikacja OpenAPI (Swagger UI)	12
6.3	Paginacja listy wniosków	13
7	Bezpieczeństwo	13
7.1	Uwierzytelnianie	13
7.2	Autoryzacja oparta na rolach	14
7.3	Zabezpieczenia formularzy	14
7.4	Dostęp publiczny do słowników	14
7.5	Optymistyczna kontrola współbieżności	14
8	Frontend i dostępność	14
8.1	Dostępność	15
9	Testy wydajnościowe	15
9.1	Testy bazodanowe (SQL)	15
9.2	Testy endpointów REST (k6) - punkt dodatkowy	15

10 Uruchomienie systemu	16
10.1 Wymagania	16
10.2 Krok po kroku	17
10.3 Konta testowe	17
11 Podsumowanie	18

1 Wstęp

1.1 Cel projektu

Celem projektu jest dostarczenie systemu informatycznego usprawniającego proces składania, oceny i rozliczania wniosków o finansowanie działalności studenckiej i doktoranckiej w Politechnice Białostockiej. System realizuje papierowy obieg wniosku opisany w *Zarządzeniu nr 13/2025 Rektora Politechniki Białostockiej z dnia 14 lutego 2025 r. w sprawie wydatkowania środków finansowych przeznaczonych na działalność studencką i doktorancką*, ze szczególnym uwzględnieniem *Załącznika nr 1* (wzór formularza wniosku).

1.2 Zakres

System obejmuje:

- rejestrację oraz logowanie użytkowników z rozróżnieniem ról,
- składanie wniosku w wersji elektronicznej w pełni odwzorowującej Załącznik nr 1,
- walidację formularza (pola obowiązkowe, spójność dat i kwot, suma źródeł finansowania),
- obieg statusów wniosku (maszyna stanów po stronie bazy danych),
- podgląd statusu i szczegółów wniosku przez wnioskodawcę,
- REST API jako warstwę komunikacji frontend–backend,
- dokumentację OpenAPI dla wystawionych usług,
- logikę składowaną w bazie danych (pakiety, widoki, wyzwalacze, funkcje),
- zgodność z wymaganiami dostępności (deklaracja dostępności).

1.3 Zespół i podział pracy

- **Jakub Matusiewicz** - warstwa bazy danych (Oracle, PL/SQL, migracje Flyway), frontend (Thymeleaf, Bootstrap), integracja widoków z REST API.
- **Jakub Borkowski** - backend Spring Boot, model encji JPA, use case'y, REST API, bezpieczeństwo (Spring Security + JWT).

2 Wymagania

2.1 Wymagania funkcjonalne

1. Rejestracja i logowanie użytkownika.
2. Wypełnianie wniosku poprzez formularz odwzorowujący Załącznik nr 1.
3. Walidacja formularza.
4. Wysyłanie wniosku do wyższych jednostek (zmiana statusu DRAFT → SUBMITTED)
- dostępne tylko od strony back-end.

5. Kontrola wniosku finansowego poprzez sprawdzanie jego statusu.
6. Edycja/poprawa wniosku w razie odrzucenia lub uwag - dostępne tylko od strony back-end.
7. Procedowanie wniosku przez wyższe jednostki (Dziekanat, CSSDiR, Kwestor) - pola dostępne dla odpowiednich ról oraz odpowiednie ścieżki dla konkretnych wniosków - dostępne tylko od strony back-end.
8. Dostępność aplikacji dla osób ze specjalnymi potrzebami.
9. Filtrowanie i wyszukiwanie wniosków - dostępne tylko od strony back-end.
10. Zarządzanie uprawnieniami i rolami użytkowników (administrator) - dostępne tylko od strony back-end.
11. Wyświetlanie pełnej historii zmian statusu wniosku - dostępne tylko od strony back-end.
12. Źródło finansowania jako zmienna w encji wniosku.

2.2 Wymagania niefunkcjonalne

- **Architektura:** klient-serwer; frontend i backend to *osobne moduły Maven* uruchamiane jako niezależne procesy Spring Boot (frontend port 8081, backend port 8080). Komunikacja wyłącznie poprzez REST API z tokenem JWT.
- **Wydajność:** poprawna praca przy zbiorach 1 000–1 000 000 wniosków (z paginacją i odpowiednimi indeksami).
- **Bezpieczeństwo:** uwierzytelnianie tokenem JWT przechowywanym w bezpiecznym ciasteczku `HttpOnly`, autoryzacja oparta na rolach.
- **Dostępność:** zgodność z WCAG 2.1 - skip link, semantyczne nagłówki, kontrast, etykiety formularzy, opublikowana deklaracja dostępności.

3 Architektura systemu

System jest aplikacją o architekturze **multi-module** – frontend i backend są *osobnymi modułami Maven*, uruchamianymi jako dwa niezależne procesy Spring Boot na różnych portach. To fizyczna realizacja wymagania „frontend działa niezależnie od backendu” z deklaracji projektowej.

- **Moduł frontend** (port 8081) – aplikacja Spring Boot z kontrolerami Spring MVC zwracającymi widoki Thymeleaf, spakowana jako uruchamialny JAR (**finansista-frontend**). Frontend *nie korzysta* z warstwy biznesowej ani repozytoriów backendu – komunikuje się z nim wyłącznie przez HTTP poprzez `RestClient` (klasa `RestClientConfig`, `baseUrl app.backend.url`), przekazując token JWT pobrany z ciasteczka przeglądarki jako nagłówek `Authorization: Bearer`.

- **Moduł backend** (port 8080) – aplikacja Spring Boot wystawiająca REST API pod `/api/v1/**` (JSON). Zawiera całą logikę domenową (use case'y), warstwę dostępu do bazy (Spring Data JPA), bezpieczeństwo (Spring Security + JWT) oraz specyfikację OpenAPI (Swagger UI).
- **Warstwa danych** – Oracle Database 23ai Free z migracjami Flyway, logiką składowaną (PL/SQL: pakiety, widoki, wyzwalacze) i dostępem przez JPA/Hibernate.

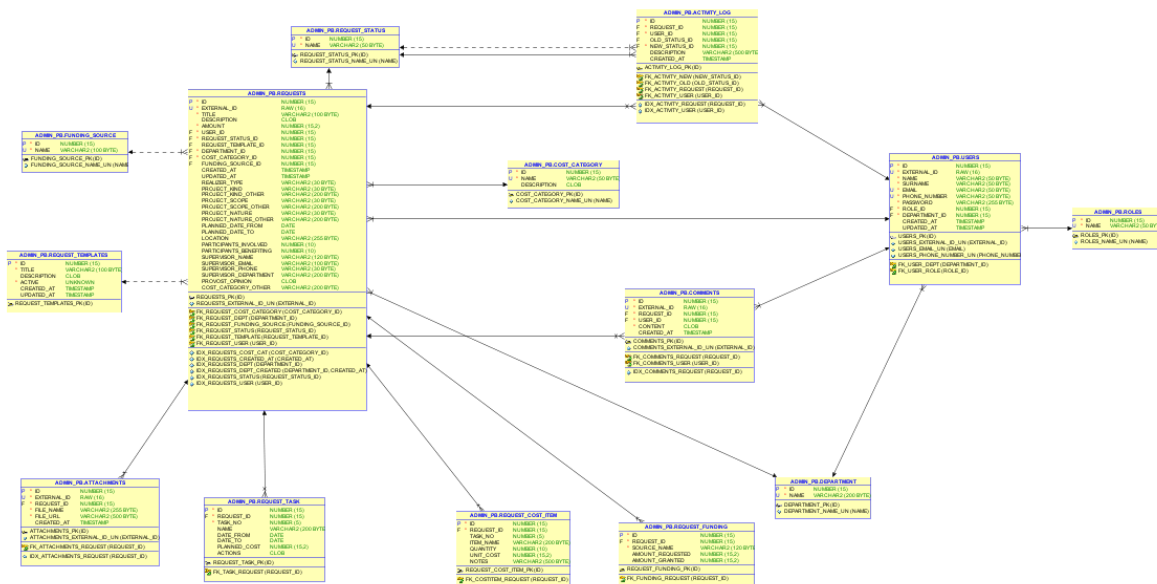
Struktura Maven Root projektu (`pom.xml` w katalogu głównym) ma typ `parent` i agreguje dwa moduły: `<module>backend</module>` i `<module>frontend</module>`. Frontend ma w `pom.xml` zależność na `finansista-backend` oznaczoną komentarzem „*TYM-CZASOWE*” – pozwala współdzielić klasy DTO (`CreateRequestRequest`, `RequestResponse` itp.) zamiast je duplikować. Docelowo planowane wydzielenie wspólnych DTO do trzeciego modułu `common` lub kontraktów generowanych z OpenAPI – po osiągnięciu czego frontend będzie mógł stać się w pełni niezależnym projektem (np. aplikacją SPA).

3.1 Stos technologiczny

- Java 25, Spring Boot 4.0
- Spring MVC, Thymeleaf 3, Bootstrap 5
- Spring Security, JWT 0.12
- Hibernate / Spring Data JPA
- Flyway (migracje)
- Oracle Database 23 Free (kontener Docker)
- Lombok, springdoc-openapi 2.8.9 (Swagger UI / OpenAPI 3.1)
- Build: Maven

4 Model danych

4.1 Diagram encji



Rysunek 1: Diagram encji

4.2 Główne tabele

- **users** - konta użytkowników; powiązane z **roles** i **department**.
- **requests** - główna encja wniosku (sekcje I–III Załącznika nr 1). Tabela zawiera komplet pól Załącznika (typ realizatora, rodzaj/zakres/charakter projektu wraz z polami „inne”, daty planowanego przedsięwzięcia, liczby uczestników, dane opiekuna naukowego).
- **request_task** - pozycje harmonogramu (sekcja IV).
- **request_cost_item** - pozycje kosztorysu (sekcja IV).
- **request_funding** - **multi-source funding**: wniosek może być finansowany z *wielu* źródeł równocześnie (Samorząd, Inicjatywy, Środki Wydziału. . .), każde z osobną kwotą wnioskowaną (**amount_requested**) i przyznaną (**amount_granted**). Para (**request_id**, **funding_source_id**) jest unikalna. Wcześniejsze pole **funding_source_id** na tabeli **requests** zostało usunięte na rzecz tej relacji.
- **request_status** - słownik statusów wniosku.
- **activity_log** - historia zmian statusu (zasilana przez trigger **T_ACTIVITY**).
- **comments**, **attachments** – komentarze i załączniki do wniosków.
- Słowniki: **roles**, **department**, **cost category**, **funding source**, **request templates**.

4.3 Wartości osadzone (*JPA Embeddables*)

Pola Załącznika nr 1 fizycznie żyją w tabeli `requests` (płaski schemat), ale w warstwie obiektowej zostały zgrupowane w **value objects** przy pomocy adnotacji `@Embeddable` – wzorzec znany z *Domain-Driven Design*. Zapewnia to czytelność modelu (mniej pól w `Request`, jasna semantyka), zachowując wydajność (brak dodatkowych JOIN-ów):

- `ProjectDetails` - sekcja I Załącznika: `realizerType`, `projectKind/projectKindOther`, `projectScope/projectScopeOther`, `projectNature/projectNatureOther`, `plannedDateFrom/plannedDateTo`, `location`, `participantsInvolved`, `participantsBenefiting`.
- `SupervisorInfo` – sekcja III Załącznika: dane opiekuna merytorycznego (imię i nazwisko, e-mail, telefon, jednostka).

Klasy te są niemutowalne (`@NoArgsConstructor(access = PROTECTED)` + konstruktor z wszystkimi polami), eksponują jedynie gettery i mają fabrykę `empty()` dla wniosku w trakcie tworzenia.

4.4 Klucze hybrydowe

Każda eksponowana encja używa dwóch identyfikatorów: wewnętrznego `id` (`NUMBER IDENTITY`) używanego w kluczach obcych oraz zewnętrznego `external_id` (`RAW(16)` – `UUID v7`), który jest jedynym identyfikatorem wystawianym w REST API i adresach URL. Zapobiega to wycieksowi informacji o liczności rekordów i utrudnia enumerację zasobów.

4.5 Migracje Flyway

Pierwotne migracje V8 (pola Załącznika 1) oraz V9 (jednostka *Samorząd Doktorantów*) zostały *zwinięte do baseline'u* (V1/V2) w ramach refaktoryzacji PR #12. W bazie pozostaje spójny ciąg migracji od V1 do V8:

- V1 – inicjalny schemat: 11 tabel (`users`, `roles`, `department`, `requests` z pełnym Załącznikiem nr 1, `request_task`, `request_cost_item`, `request_funding`, `request_status`, `request_templates`, `cost_category`, `funding_source`, `comments`, `attachments`, `activity_log`).
- V2 – wypełnienie słowników: 7 ról, 15 wydziałów (w tym Samorząd Studentów i Samorząd Doktorantów), 7 statusów, 2 szablony wniosków, 2 kategorie kosztów, 4 źródła finansowania.
- V3 – widoki bazodanowe (`v_department_requests_summary`, `v_request_details`).
- V4 – wyzwalacze (`T_REQUESTS_UPDATE_NOW`, `T_ACTIVITY` audytowy, `T_BLOCK` blokada edycji wysłanych wniosków).
- V5 – pakiet PL/SQL `finansista_pkg` (`set_actor/get_actor`, `department_used_budget`, `evaluate_request`).
- V6 – indeksy wydajnościowe.
- V7 – maszyna stanów wniosku (funkcja walidująca + trigger blokujący niedozwolone przejścia, ORA-20002).

- V8 – skrypt zasilania danymi demonstracyjnymi (6 użytkowników z różnymi rolami, 3 przykładowe wnioski w różnych statusach, wpisy do `request_funding`). Wszystkie wstawki idempotentne (`INSERT ...SELECT FROM dual WHERE NOT EXISTS`) – skrypt może zostać uruchomiony wielokrotnie bez naruszenia ograniczeń `UNIQUE`.

5 Logika w bazie danych

5.1 Maszyna stanów wniosku

Dozwolone przejścia statusów są egzekwowane na poziomie bazy (V7):

Status	Dozwolone przejścia
DRAFT	SUBMITTED
SUBMITTED	FORMAL_EVALUATION, REJECTED, CORRECTION_REQUIRED
FORMAL_EVALUATION	UNDER_REVIEW, REJECTED, CORRECTION_REQUIRED
UNDER_REVIEW	ACCEPTED, REJECTED, CORRECTION_REQUIRED
CORRECTION_REQUIRED	DRAFT, SUBMITTED

Próba zmiany statusu na niedozwolony skutkuje błędem `ORA-20002`.

5.2 Pakiet `finansista_pkg` (V5)

Logika domenowa wymagająca dostępu do wielu tabel lub stanu sesyjnego została zamknięta w pakiecie PL/SQL `finansista_pkg`. Pakiet eksponuje trzy publiczne API:

- PROCEDURE `set_actor(p_user_id)` oraz FUNCTION `get_actor` RETURN NUMBER - przechowują w zmiennej pakietowej (*per-session state*) identyfikator użytkownika, który wykonuje aktualnie operację. Aplikacja Spring wywołuje `set_actor` przed każdą zmianą statusu, dzięki czemu trigger audytowy zapisuje w `activity_log` rzeczywistego *aktora* (np. pracownika Dziekanatu), a nie wnioskodawcę.
- FUNCTION `department_used_budget(p_dept_id)` RETURN NUMBER – pole pochodne: zwraca łączną kwotę wniosków zaakceptowanych (`status_id = 5`) dla danego wydziału. Używana w raportach bez konieczności ręcznego pisania agregacji po stronie aplikacji.
- PROCEDURE `evaluate_request(p_req_id, p_evaluator_id, p_new_status_id, p_comment_con` - atomowa procedura oceny wniosku: ustawia aktora, blokuje wiersz (`SELECT ...FOR UPDATE`), weryfikuje że wniosek jest w fazie weryfikacji (`SUBMITTED / FORMAL_EVALUATION / UNDER_REVIEW`, w przeciwnym razie `ORA-20003`), zmienia status i opcjonalnie dopisuje komentarz. Procedura świadomie **nie** wykonuje `COMMIT` – transakcję zatwierdza warstwa aplikacji (`@Transactional`), co pozwala wycofać całość w razie błędu po stronie Javy.

5.3 Triggery (V4)

Na tabeli `requests` założono trzy triggery wierszowe:

- `T_REQUESTS_UPDATE_NOW` (*BEFORE UPDATE*) - automatycznie ustawia `updated_at = CURRENT_TIMESTAMP`, gwarantując że pole nie zostanie pominięte przy zapomnianej aktualizacji w kodzie aplikacji.

- **T_ACTIVITY** (*AFTER UPDATE OF request_status_id*, WHEN NEW != OLD) - audyt zmian statusu. Wstawia do `activity_log` rekord ze starym i nowym statusem oraz aktorem pobranym z `finansista_pkg.get_actor` (z fallbackiem na wnioskodawcę, gdyby aplikacja nie ustawiła aktora).
- **T_BLOCK** (*BEFORE UPDATE*) - zabezpieczenie integralności: jeśli wniosek nie jest już w statusie DRAFT ani CORRECTION_REQUIRED, blokuje zmianę pól treściowych (`title`, `amount`, `description`, `cost_category_id`) - próba kończy się błędem ORA-20001. Porównanie pól CLOB odbywa się przez `DBMS_LOB.COMPARE` z obsługą NULL (`NVL` na `EMPTY_CLOB()`).

5.4 Maszyna stanów (V7)

Funkcja `validate_request_status_change(p_old, p_new)` oraz trigger `T_REQUESTS_TRANSITION` blokują niedozwolone przejścia statusów (tabela dozwolonych przejść w sekcji 5, podsekcja *Maszyna stanów wniosku*). Każda nielegalna zmiana statusu kończy się błędem ORA-20002 podnoszonym po stronie bazy, przez co aplikacja nie musi (i nie może) tej zasady obejść.

5.5 Widoki (V3)

- **v_department_requests_summary** - podsumowanie ilościowe i kosztowe wniosków w rozbiciu na wydziały. Dla każdego wydziału zwraca: łączną liczbę wniosków i ich łączną kwotę, liczbę i sumę wniosków zaakceptowanych (`status_id = 5`) oraz odrzuconych (`status_id = 6`). Sumowanie warunkowe jest zrealizowane przez `SUM(CASE WHEN ...)`, a `LEFT JOIN` z tabelą `department` zapewnia, że na liście pojawiają się też wydziały bez żadnego wniosku (z zerowymi licznikami). Wykorzystywany w raporcie administracyjnym.
- **v_request_details** - denormalizacja na potrzeby listingu wniosków: każdy wiersz to wniosek wzbogacony o czytelne nazwy powiązanych słowników (autor jako `name || ' ' || surname`, nazwa wydziału, nazwa kategorii kosztu, nazwa statusu). Pozwala zwrócić tabelę listingową jednym zapytaniem zamiast czterech JOIN-ów w warstwie aplikacji.

5.6 Indeksy wydajnościowe (V6)

Po stronie bazy założono indeksy na kolumnach najczęściej używanych w filtrach i sortowaniach listingów wniosków:

- **IDX_REQUESTS_STATUS** na `request_status_id` - filtrowanie po statusie (np. „wnioski do oceny”),
- **IDX_REQUESTS_USER** na `user_id` - listing własnych wniosków wnioskodawcy,
- **IDX_REQUESTS_CREATED_AT** na `created_at` DESC - domyślne sortowanie listingu od najnowszych,
- **IDX_REQUESTS_DEPT_CREATED** na (`department_id`, `created_at` DESC) - indeks złożony pod najczęstsze zapytanie: „wnioski mojego wydziału, posortowane od najnowszych”.

5.7 Wyniki testów wydajnościowych (1 k – 1 M)

Testom poddano tabelę transakcyjną `requests` dla skali **1 000 / 10 000 / 100 000 / 1 000 000** wierszy. Środowisko: Oracle Database 23ai Free (Docker `gvenzl/oracle-free`), schemat `ADMIN_PB`. Dane testowe generowane skryptem `db/scripts/demo_data_load.sql` (set-based `INSERT ...SELECT` z `CONNECT BY`, kwoty z `DBMS_RANDOM`, równomierny rozkład po użytkownikach/statusach/wydziałach). Po każdym zasileniu odświeżano statystyki optymalizatora (`DBMS_STATS.GATHER_TABLE_STATS`). Pomiar czasu: `SET TIMING ON` w `SQL*Plus`.

Mierzone zapytania

Nr	Zapytanie	Scenariusz w aplikacji
Q1	<code>ORDER BY created_at DESC OFFSET 40 FETCH NEXT 20</code>	paginacja listy wniosków (3. strona)
Q2	<code>WHERE department_id = 1 ORDER BY created_at DESC FETCH 20</code>	przegląd wniosków własnego wydziału
Q3	<code>SELECT * FROM v_department_requests_summary</code>	raport admina: ilości i koszty per wydział
Q4	<code>WHERE external_id = :uuid</code>	pobranie wniosku po identyfikatorze z API / linku

Czasy wykonania

Zapytanie	1 tys.	10 tys.	100 tys.	1 mln
Q1 – paginacja listy	0,01 s	0,01 s	0,04 s	0,27 s
Q2 – filtr wydziału + sort	0,01 s	0,00 s	0,03 s	0,17 s
Q3 – raport agregowany	0,01 s	0,01 s	0,03 s	0,31 s
Q4 – wyszukanie po UUID	0,00 s	0,00 s	0,00 s	0,00 s

Plany wykonania (skala 1 mln) Najważniejsze potwierdzenie skuteczności indeksów to plan wykonania zapytania Q2, przez co silnik wykorzystuje indeks złożony `IDX_REQUESTS_DEPT_CREATED`, sortowanie odbywa się *bez* operacji `SORT` (odczyt indeksu od końca, `DESCENDING`), a przetwarzanie przerywa się po pobraniu 20 wierszy (`WINDOW NOSORT STOPKEY`):

1	WINDOW NOSORT STOPKEY	20	12
	(0)		
2	TABLE ACCESS BY INDEX ROWID REQUESTS		
3	INDEX RANGE SCAN DESCENDING IDX_REQUESTS_DEPT_CREATED	20	3
	(0)		

Kod 1: Plan wykonania Q2 dla 1 mln wierszy

Q4 - wyszukanie po `external_id` - korzysta z unikalnego indeksu zakładanego przez ograniczenie `UNIQUE` na kolumnie:

1	TABLE ACCESS BY INDEX ROWID	REQUESTS	1	3 (0)	
2	INDEX UNIQUE SCAN	SYS_C008705	1	2 (0)	

Kod 2: Plan wykonania Q4 dla 1 mln wierszy

Wnioski

1. **Zaproponowane indeksy spełniają zadanie.** Kluczowe zapytania listingowe (Q2, Q4) działają w czasie praktycznie stałym niezależnie od skali, czyli koszt planu dla Q2 przy 1 mln wierszy to 12 jednostek (20 wierszy z indeksu), a wyszukanie po UUID to pojedynczy INDEX UNIQUE SCAN.
2. **Paginacja OFFSET/FETCH (Q1) skaluje się akceptowalnie** (0,27 s przy 1 mln), ale jej koszt rośnie z głębokością strony – baza musi policzyć i pominąć wcześniejsze wiersze. Przy bardzo głębokim przewijaniu rekomendowana technika to *keyset pagination* (`WHERE created_at < :ostatnia_data ORDER BY created_at DESC FETCH NEXT 20`), korzystająca wprost z `IDX_REQUESTS_CREATED_AT` bez operacji OFFSET.
3. **Raport agregowany (Q3) czyta pełną tabelę** - 0,31 s przy 1 mln wierszy jest akceptowalne dla raportu administracyjnego. Gdyby tabela istotnie urosła, naturalnym krokiem jest widok zmaterializowany (`MATERIALIZED VIEW`) odświeżany cyklicznie (np. co godzinę), zamiast wyliczania agregatów na żywo.
4. Wstawienie 900 tys. wierszy metodą set-based trwało pojedyncze sekundy, co potwierdza przepustowość zapisu przy ładowaniu wsadowym, a pakiet `finansista_pkg` mógłby być wykorzystywany w hurtowej migracji wniosków archiwalnych bez modyfikacji.

6 REST API

Frontend Thymeleaf komunikuje się z backendem wyłącznie poprzez REST API, co wymusza poprawną architekturę klient-serwer. Token JWT z ciasteczka przeglądarki jest przekazywany jako nagłówek `Authorization: Bearer`.

6.1 Najważniejsze endpointy

Metoda	Ścieżka	Opis
POST	/api/v1/auth/register	rejestracja konta
POST	/api/v1/auth/login	logowanie (ustawia ciasteczko JWT)
POST	/api/v1/auth/logout	wylogowanie (kasuje ciasteczko)
GET	/api/v1/requests	lista wniosków (filtr, paginacja <code>Pageable</code>)
POST	/api/v1/requests	utworzenie wniosku (pełny Załącznik nr 1)
GET	/api/v1/requests/{id}	szczegóły wniosku (zwraca ETag z wersją)
PUT	/api/v1/requests/{id}	edycja wniosku (wymaga <code>If-Match</code>)
DELETE	/api/v1/requests/{id}	usunięcie wniosku (admin)
PATCH	/api/v1/requests/{id}/status	zmiana statusu
POST	/api/v1/requests/{id}/funding	przyznanie kwoty per źródło finansowania
GET	/api/v1/requests/{id}/history	historia zmian statusu
POST	/api/v1/requests/{id}/comments	dodanie komentarza
GET	/api/v1/requests/{id}/comments	komentarze
GET	/api/v1/request-templates	lista aktywnych szablonów (publiczne)
POST/PUT/DELETE	/api/v1/request-templates/{id}	CRUD szablonów (admin)
GET	/api/v1/request-statuses	słownik statusów (publiczne)
GET	/api/v1/reference/departments	lista wydziałów (publiczne)
POST/PUT/DELETE	/api/v1/reference/departments/{id}	CRUD wydziałów (admin)
GET	/api/v1/reference/cost-categories	kategorie kosztów (publiczne)
GET	/api/v1/reference/funding-sources	źródła finansowania (publiczne)
GET	/api/v1/user/roles	słownik ról (publiczne)
POST/PUT/DELETE	/api/v1/user/roles/{id}	CRUD ról (admin)

6.2 Specyfikacja OpenAPI (Swagger UI)

W celu spełnienia wymagania „*Aplikacja typu Web API z obsługą Open API*” do projektu dołączono bibliotekę `org.springdoc:springdoc-openapi-starter-webmvc-ui` w wersji 2.8.9. Biblioteka automatycznie skanuje wszystkie kontrolery oznaczone adnotacją `@RestController`, generuje specyfikację OpenAPI 3.1 i udostępnia interaktywne UI

Swaggera bez dodatkowej konfiguracji.

Publiczne punkty dostępu (dodane do listy `permitAll` w klasie `SecurityConfig`):

- <http://localhost:8080/swagger-ui.html> - interaktywna konsola (umożliwia wykonywanie zapytań z przeglądarki),
- <http://localhost:8080/swagger-ui/index.html> - bezpośredni adres UI,
- <http://localhost:8080/v3/api-docs> - surowa specyfikacja JSON, wykorzystywana m.in. przez generatory klientów REST.

Specyfikacja obejmuje wszystkie endpointy REST w przestrzeni `/api/v1/**` - uwierzytelnianie, wnioski, komentarze, historię, szablony, statusy, role oraz słowniki referencyjne.

6.3 Paginacja listy wniosków

Endpoint `GET /api/v1/requests` zwraca dane stronicowane przy pomocy mechanizmu `Pageable` z biblioteki Spring Data. Parametry zapytania:

- `page` - numer strony (domyślnie 0),
- `size` - liczba elementów na stronie (domyślnie 1000, co zachowuje wsteczną kompatybilność z istniejącymi konsumentami REST),
- `sort` - pole sortowania (domyślnie `createdAt,desc`).

Repozytorium `SpringDataJpaRequestRepository` implementuje interfejs `JpaSpecificationExecutor` dzięki czemu paginacja działa łącznie ze specyfikacją autoryzacyjną `RequestAccessSpecificationFactory`. Użytkownik nigdy nie zobaczy wniosków poza zakresem swoich uprawnień, niezależnie od żądanej strony, przez co specyfikacja autoryzacyjna jest dołączana do każdego zapytania jako pierwszy warunek `WHERE`.

Adnotacja `@EntityGraph` (z atrybutami `status`, `department`, `costCategory`, `fundingSource`, `template`) jest powielona na obu wariantach `findAll`, co eliminuje problem N+1 na obu ścieżkach.

W odpowiedzi HTTP zwracane są dodatkowe nagłówki:

- `X-Total-Count` - łączna liczba pasujących wniosków,
- `X-Total-Pages` - liczba dostępnych stron.

Ciało odpowiedzi pozostaje płaską listą `List<RequestResponse>` (uzyskaną przez `page.getContent()`), co zachowuje wsteczną zgodność kontraktu REST z istniejącą warstwą frontową.

7 Bezpieczeństwo

7.1 Uwierzytelnianie

Po poprawnym zalogowaniu backend wystawia token JWT podpisany kluczem HMAC SHA-256 i zwraca go w bezpiecznym ciasteczku z atrybutami `HttpOnly`, `SameSite=Strict`. Token zawiera identyfikator użytkownika (e-mail) oraz jego rolę.

7.2 Autoryzacja oparta na rolach

- `ROLE_STUDENT` - składa wnioski, widzi własne;
- `ROLE_WRSS` - ocena merytoryczna w ramach Samorządu;
- `ROLE_LEGAL_COMMISSION` - ocena merytoryczna w komisji;
- `ROLE_DEAN_OFFICE` - ocena formalna (CSSDiR/Dziekanat);
- `ROLE_FINANCE_OFFICE` - Kwestura (rozliczenie);
- `ROLE_ADMIN` – decyzja Rektora, zarządzanie systemem.

7.3 Zabezpieczenia formularzy

Rola użytkownika *nigdy* nie jest pobierana z formularza rejestracji, tzn. każde nowe konto otrzymuje rolę bazową `ROLE_STUDENT` wymuszoną po stronie serwera. Przypisanie wyższej roli jest wyłącznie aktem administracyjnym. Podobnie wydział wnioskodawcy nie jest wybierany przy składaniu wniosku, a pobierany jest z konta zalogowanego użytkownika.

7.4 Dostęp publiczny do słowników

Po refaktoryzacji PR #12 endpointy GET dla danych referencyjnych (`/api/v1/reference/departments`, `/api/v1/reference/cost-categories`, `/api/v1/reference/funding-sources`, `/api/v1/user/roles`, `/api/v1/request-templates`, `/api/v1/request-statuses`) są dostępne anonimowo. Umożliwia to zasilenie list rozwijanych w formularzu rejestracji (np. wybór wydziału) bez konieczności wcześniejszego logowania. Operacje modyfikujące (POST/PUT/DELETE) na tych samych zasobach wymagają roli `ROLE_ADMIN` (egzekwowane przez `@PreAuthorize`).

7.5 Optymistyczna kontrola współbieżności

Edycja wniosku oraz zmiana statusu wymagają nagłówka `If-Match` z bieżącą wersją encji (zwracaną w nagłówku `ETag` poprzedniego GET). Mechanizm jest realizowany przez:

- kolumnę `version` (`@Version` w JPA) na encjach `Request` i `User`,
- helper `ETags` parsujący/formatujący wartość `ETag`,
- `assertVersion` guard w use case'ach `EditRequestUseCase` i `ChangeRequestStatusUseCase` – niezgodność wersji kończy się odpowiedzią HTTP 412 `Precondition Failed`, zły format nagłówka – 400 `Bad Request` przez `InvalidIfMatchHeaderException`.

Zapobiega to klasycznemu wyścigowi, gdy dwóch pracowników jednocześnie ocenia ten sam wniosek (deklaracja: „współbieżność bazy danych”).

8 Frontend i dostępność

Warstwa prezentacji oparta jest o szablony Thymeleaf z motywem Politechniki Białostockiej (kolorystyka zielona, flat design, Bootstrap 5). Każdy widok dziedziczy po wspólnym układzie `layout/base.html` zawierającym nawigację i stopkę.

8.1 Dostępność

- Skip link na początku każdego widoku.
- Semantyczne nagłówki i etykiety formularzy.
- Kontrast kolorów zgodny z WCAG 2.1 AA.
- Opublikowana *Deklaracja dostępności* pod adresem `/accessibility`.
- Strony błędów 400/403/404/500 w spójnym stylu HTML (nie JSON).

9 Testy wydajnościowe

9.1 Testy bazodanowe (SQL)

Wykonano testy wydajnościowe na zbiorach 1 000, 10 000, 100 000 oraz 1 000 000 wierszy w tabeli `requests`. Sprawdzano:

- czas pobrania pojedynczego wniosku po `external_id`,
- czas listowania wniosków z paginacją,
- czas zmiany statusu (kluczowa operacja transakcyjna),
- plany wykonania zapytań przed i po dodaniu indeksów (V6).

Szczegółowe wyniki oraz porównanie planów wykonania zostały umieszczone w pliku `docs/testy-wydajnosciowe.md`.

9.2 Testy endpointów REST (k6) - punkt dodatkowy

Uzupełnieniem testów SQL są testy całych ścieżek HTTP, weryfikujące zachowanie warstwy aplikacji (Spring Boot + JPA + serializacja JSON) pod realnym, równoległym obciążeniem klientów. Wykorzystano narzędzie **k6** firmy Grafana Labs (skrypt `docs/k6/get-requests.k6.js`).

Profil obciążenia Symulacja narastającego ruchu w czterech fazach:

1. rozgrzewka - ramp-up do **100 VU** przez 30 s,
2. narastanie - do **500 VU** przez 1 min,
3. szczyt - **1000 VU** utrzymywane przez 2 min,
4. wygaszanie - ramp-down do 0 VU przez 30 s.

Kryteria akceptacji Progi thresholds k6: `p95 < 800 ms`, `p99 < 1500 ms`, poziom błędów `< 1%`. Ich przekroczenie generuje exit code różny od zera i wpis `ERROR` w stdout, dzięki czemu identyfikacja wąskich gardeł jest obiektywna.

Wyniki dla 1000 VU (test trwał 4 min) Test powtórzono dla czterech wartości parametru zapytania `size`, żeby zweryfikować skuteczność paginacji.

size	żądań	avg	p95	p99	HTTP fail	checks
10	74 700	675 ms	1481 ms	1624 ms	0%	4%
50	75 145	665 ms	1477 ms	1628 ms	0%	4%
500	71 852	742 ms	1599 ms	1727 ms	0%	9%
1000	72 438	730 ms	1777 ms	–	0%	9%

Analiza wąskich gardeł

1. **Warstwa HTTP jest stabilna** - współczynnik `http_req_failed` wyniósł 0% we wszystkich przebiegach. Backend nie zwrócił żadnego błędu 5xx, nie wystąpiły timeouty ani zerwane połączenia.
2. **Rozmiary stron 10 i 50 dają niemal identyczne wyniki** (avg 675 vs 665 ms, p95 1481 vs 1477 ms). Oznacza to, że dominującym kosztem nie jest serializacja JSON ani rozmiar payloadu, lecz koszt stały per-request. Główne podejrzenia: pula HikariCP (domyślnie 10) wysycona przez 1000 współbieżnych żądań, narzut filtra JWT (parsowanie tokena), koszt deserializacji `RequestResponse` z zagnieżdżonymi listami.
3. **Skok kosztów przy size=500 i size=1000** (błędy 4% → 9%, p95 +20%) - dopiero duże strony zaczynają obciążać CPU serializacją i pamięć (większy working set per request). Wynik potwierdza tezę, że rekomendowana domyślna strona dla tego endpointu to `size ≤ 50`.
4. **Próg p95 < 800 ms jest nieosiągalny przy 1000 VU na tym sprzęcie** niezależnie od rozmiaru strony. Realny SLO dla aplikacji uniwersyteckiej (kilkaset jednoczesnych użytkowników) p95 < 500 ms przy `size ≤ 50` wymagałby zwiększenia rozmiaru puli HikariCP do ~50 oraz rozważenia *keyset pagination* dla głębokich stron (analogicznie do rekomendacji z testów SQL).
5. **Przepustowość ustabilizowała się na ~300 RPS** niezależnie od `size` (71-75 tys. żądań / 4 min). Bezpośrednio potwierdza to, że wąskim gardłem jest dostępność wątku/połączenia, a nie objętość pojedynczej odpowiedzi.

Szczegółowy raport (czterokrotny przebieg, pełne podsumowania w JSON oraz przepis odtworzenia) znajduje się w pliku `docs/testy-wydajnosciowe.md` oraz `docs/k6/get-requests.summary`.

10 Uruchomienie systemu

10.1 Wymagania

- Java 25, Maven 3.9+
- Docker (do uruchomienia Oracle Database 23 Free)

10.2 Krok po kroku

```
1 # 1. Uruchomienie bazy Oracle w Dockerze
2 docker run -d --name oracle-pb -p 1521:1521 \
3   -e ORACLE_PASSWORD=pawlewjavle \
4   gvenzl/oracle-free:23-slim
5
6 # 2. Utworzenie schematu admin_pb (z hasłem zgodnym z application.yaml)
7 #   - skrypt w docs/install/create_schema.sql lub ręcznie w SQL
8   Developer
9
10 # 3. Zbudowanie obu modułów (uruchamia Flyway na backendzie)
11 ./mvnw clean install -DskipTests
12
13 # 4. Uruchomienie BACKENDU (REST API, port 8080) - w osobnym terminalu
14 cd backend && ../mvnw spring-boot:run
15
16 # 5. Uruchomienie FRONTENDU (Thymeleaf, port 8081) - w drugim terminalu
17 cd frontend && ../mvnw spring-boot:run
18
19 # 6. Aplikacja dostępna pod:
20 #   - frontend (UI):      http://localhost:8081
21 #   - backend (REST API): http://localhost:8080/api/v1/...
```

Kod 3: Uruchomienie aplikacji (multi-module)

10.3 Konta testowe

Aplikacja przygotowuje konta demonstracyjne z dwóch niezależnych źródeł. Oba źródła współistnieją bez konfliktu: DataSeeder sprawdza `existsByEmail` przed zapisem, a migracja V8 używa warunku `WHERE NOT EXISTS` – każde uruchomienie jest idempotentne.

Konta z DataSeeder.java (warstwa aplikacji) Tworzone w runtime przy starcie aplikacji, hasła hashowane BCryptem przez PasswordEncoder.

E-mail	Hasło	Rola / jednostka
j.wnioskodawca@student.pb.edu.pl	student123	STUDENT / Wydział Informatyki
k.samorzad@pb.edu.pl	wrss123	WRSS / Samorząd Studentów
a.zgodna@pb.edu.pl	komisja123	LEGAL_COMMISSION / Wydział Mechaniczny
e.dziekan@pb.edu.pl	dziekanat123	DEAN_OFFICE / Dziekanat WI
j.matusiewicz@student.pb.edu.pl	admin123	ADMIN / Wydział Mechaniczny
j.borkowski@student.pb.edu.pl	admin123	ADMIN / Wydział Informatyki

Konta z migracji V8__demo_data.sql (warstwa bazy) Wstawiane przez Flyway w ramach skryptu zasilania danymi demo. Hasła zapisane w formacie `{noop}` (rozpoznawany przez DelegatingPasswordEncoder jako plain text – świadoma decyzja, by nie liczyć BCryptu po stronie SQL). Identyfikatory wewnętrzne (`id`) zaczynają się od 9001, aby nie kolidować z auto-inkrementacją kont z DataSeeder.

E-mail	Hasło	Rola / jednostka
demo-admin@pb.edu.pl	demo123	ADMIN / Wydział Informatyki
demo-student@pb.edu.pl	demo123	STUDENT / Wydział Informatyki
demo-wrss@pb.edu.pl	demo123	STUDENT_COUNCIL / Samorząd Studentów
demo-komisja@pb.edu.pl	demo123	LEGAL_COMMISSION / Rektorat
demo-dziekanat@pb.edu.pl	demo123	DEAN_OFFICE / Dziekanat WI
demo-kwestura@pb.edu.pl	demo123	FINANCE_OFFICE / Kwestura

Przykładowe wnioski z V8 Razem z kontami migracja V8 wstawia trzy wnioski demonstracyjne (z odpowiadającymi rekordami w `request_funding`, demonstrując multi-source finansowanie), po jednym w różnym statusie obiegu:

- *Juwenalia 2026* - status DRAFT, autor: demo-student, kwota 12 500,00 zł,
- *Konferencja koła naukowego* - status SUBMITTED, autor: demo-student, kwota 4 800,00 zł,
- *Hackathon studencki* - status ACCEPTED, autor: demo-wrss, kwota 7 600,00 zł.

11 Podsumowanie

Projekt **Finansista PB** z powodzeniem zrealizował kluczowe założenia architektoniczne, koncentrując się przede wszystkim na solidnej warstwie serwerowej oraz zaawansowanej logice bazodanowej. Udało się zaimplementować wydajne REST API, zintegrowaną maszynę stanów w Oracle PL/SQL, a także kompleksowe mechanizmy autoryzacji oparte na rolach, jednakże ze względu na wysoki stopień skomplikowania backendu i ograniczone ramy czasowe, warstwa prezentacji (frontend) została zrealizowana w wariantcie podstawowym, co stanowi obecnie fundament wymagający dalszej rozbudowy o brakujące widoki i pełne ścieżki użytkownika. Mimo tych braków interfejsowych, realizacja systemu pozwoliła na zdobycie cennego, praktycznego doświadczenia z zakresu optymalizacji zapytań SQL, zarządzania transakcjami oraz integracji nowoczesnego frameworka Spring Boot z relacyjną bazą danych.